CVM++

Stack-Based Virtual Machine & Custom Compiler

Project Report

Course	Systems Programming / Compiler Design
Language	C++17
Paradigm	Stack-Based Virtual Machine
File Extension	.cvm (CVM scripting language)

1. Introduction

Most developers use high-level languages like Python, JavaScript, or Java without ever deeply understanding how raw text is translated into instructions that a computer can execute. CVM++ demystifies this process by building a complete language pipeline entirely from scratch in modern C++17.

The project implements a lightweight custom scripting language called CVM, which compiles source code down to a proprietary bytecode format. This bytecode is then executed by a custom-built, stack-based Virtual Machine (VM). Every component — Lexer, Parser, Compiler, and VM — is implemented without any external libraries.

1.1 Problem Statement

The gap between high-level source code and machine execution is often treated as a black box. This project aims to open that black box by constructing every layer of the execution pipeline from first principles, providing hands-on understanding of compiler theory, bytecode design, and virtual machine architecture.

2. Goals

Design a Language Grammar & ISA: Define a simple but complete grammar for the CVM scripting language and a corresponding Instruction Set Architecture (ISA) for the VM.

Build a Lexer: Convert raw source code strings into a flat stream of typed Tokens (e.g., NUMBER, PLUS, IDENTIFIER, LET, WHILE).

Build a Parser: Use recursive descent parsing to arrange Tokens into a hierarchical Abstract Syntax Tree (AST) that represents program structure.

Implement a Bytecode Compiler: Walk the AST and flatten it into a compact array of raw bytecode instructions (Opcodes) stored as std::vector<uint8_t>.

Build a Virtual Machine: Execute the bytecode using a stack-based execution loop with an operand stack, variable store, and instruction pointer.

Provide a CLI Interface: Deliver both a REPL (interactive mode) and a file runner, plus a debug mode that dumps tokens, AST, and bytecode.

3. System Architecture

CVM++ implements a classic four-stage compilation and execution pipeline. Each stage is implemented as an independent module with a clean interface:

Stage	Module	Input	Output
1. Lexical Analysis	Lexer	Raw source string (.cvm)	Token stream
2. Parsing	Parser	Token stream	Abstract Syntax Tree (AST)
3. Code Generation	Compiler	AST	Bytecode (uint8_t array)
4. Execution	Virtual Machine	Bytecode	Program output

3.1 Lexer

The Lexer performs lexical analysis (tokenization). It scans the source string character by character, recognizing patterns and emitting typed Token objects. It handles whitespace skipping, integer literals, identifiers, keywords (let, if, else, while, print, input, true, false), and all operators and delimiters. Unrecognized characters produce a lexer error.

3.2 Parser

The Parser uses recursive descent to consume the token stream and construct an Abstract Syntax Tree (AST). Each grammar rule maps to a parsing function. The AST uses std::unique_ptr for memory-safe node ownership. Node types include: ProgramNode, LetNode, AssignNode, PrintNode, InputNode, IfNode, WhileNode, BinaryOpNode, NumberLiteralNode, BoolLiteralNode, and IdentifierNode.

3.3 Bytecode Compiler

The Compiler walks the AST using the Visitor pattern and emits bytecode instructions into a std::vector<uint8_t>. It maintains a symbol table mapping variable names to integer indices. Multi-byte operands (int64_t for integers, int32_t for jump offsets) are written in little-endian byte order. Jump targets are resolved using backpatching.

3.4 Virtual Machine

The VM is a stack-based execution engine. It maintains an operand stack (std::vector<Value>), a variables array (std::vector<Value>), and an instruction pointer (size_t ip). The main run() loop dispatches on the current opcode, performing push/pop operations, arithmetic, comparisons, jumps, I/O, and halting.

4. CVM Language Specification

4.1 Data Types

Type	Examples	Description
Integer	0, 42, -7, 1000	64-bit signed integer (int64_t)
Boolean	true, false	1-byte boolean value

4.2 Syntax Examples

Variables:

```
let x = 10;
let y = x + 5;
x = x * 2;
```

Arithmetic:

```
print 6 * 7;
print 100 / 4;
print 10 - 3;
```

If / Else:

```
if (x < 10) {
    print x;
} else {
    print 0;
}
```

While Loop:

```
let i = 0;
while (i < 5) {
    print i;
    i = i + 1;
}
```

Input / Output:

```
input x;
let doubled = x * 2;
print doubled;
```

5. Instruction Set Architecture (ISA)

Each instruction is identified by a 1-byte opcode. Multi-byte operands follow immediately in the bytecode stream. The VM reads them using pointer arithmetic on the bytecode array.

Opcode	Operand Bytes	Description
PUSH_INT	8 (int64_t)	Push a 64-bit signed integer constant onto the stack
PUSH_BOOL	1 (uint8_t)	Push a boolean value (0=false, 1=true) onto the stack
LOAD_VAR	1 (uint8_t)	Push the value of variable[index] onto the stack
STORE_VAR	1 (uint8_t)	Pop stack top and store into variable[index]
ADD	0	Pop two integers, push their sum
SUB	0	Pop two integers, push their difference
MUL	0	Pop two integers, push their product
DIV	0	Pop two integers, push their quotient
EQUAL	0	Pop two values, push bool (a == b)
LESS_THAN	0	Pop two values, push bool (a < b)
PRINT	0	Pop top of stack and print to stdout
INPUT	1 (uint8_t)	Read integer from stdin, store in variable[index]
JUMP	4 (int32_t)	Unconditional jump to absolute bytecode offset
JUMP_IF_FALSE	4 (int32_t)	Pop bool; jump to offset if false
POP	0	Discard top of stack
HALT	0	Stop VM execution

6. Project Structure

File	Responsibility
`token.h`	TokenType enum and Token struct
`lexer.h / lexer.cpp`	Tokenizes source code into token stream
`ast.h`	All AST node class definitions
`parser.h / parser.cpp`	Recursive descent parser, builds AST
`bytecode.h`	OpCode enum (uint8_t values)
`value.h`	Runtime Value type (INT / BOOL union)
`compiler.h / compiler.cpp`	AST visitor, emits bytecode + symbol table
`vm.h / vm.cpp`	Stack-based execution engine
`main.cpp`	Entry point: REPL, file runner, debug mode

`CMakeLists.txt`	CMake build configuration
`examples/*.cvm`	Sample CVM language programs

7. Tech Stack

Category	Technology / Concept
Language	C++17
Build Tool	CMake 3.16+ (optional: direct g++ compile)
Parsing Technique	Recursive Descent Parsing
AST Memory	std::unique_ptr (no manual new/delete)
Bytecode Storage	std::vector<uint8_t>
VM Stack	std::vector<Value>
Key C++17 Features	std::optional, structured bindings, if-init statements
Reference	Crafting Interpreters by Robert Nystrom

8. Demo Programs

8.1 Calculator (calculator.cvm)

Reads two integers from the user and prints the result of all four arithmetic operations:

```
input a;
input b;
let sum = a + b;
let diff = a - b;
let prod = a * b;
let quot = a / b;
print sum;
print diff;
print prod;
print quot;
```

Sample Run:

```
$ ./cvm examples/calculator.cvm
10
3
13
7
30
3
```

8.2 Truth Machine (truth_machine.cvm)

A classic computer science demo: reads 0 or 1. If 0, prints 0 once. If 1, prints 1 forever:

```
input x;
if (x == 0) {
    print 0;
} else {
    while (1 == 1) {
        print 1;
    }
}
```

Sample Run (input 0):

```
$ ./cvm examples/truth_machine.cvm
0
0
```

Sample Run (input 1):

```
$ ./cvm examples/truth_machine.cvm
1
```

```
1
1
1
... (infinite)
```

9. Build and Run Instructions

Direct Compile:

```
g++ -std=c++17 -Wall -o cvm src/main.cpp src/lexer.cpp src/parser.cpp
src/compiler.cpp src/vm.cpp
```

CMake Build:

```
mkdir build && cd build
cmake ..
make
```

Run Modes:

```
./cvm examples/calculator.cvm      # Run script
./cvm                            # REPL mode
./cvm --debug examples/hello.cvm  # Debug mode (tokens + AST + bytecode)
```

10. Conclusion

CVM++ successfully implements a complete language execution pipeline from scratch in C++17. The project demonstrates a working Lexer, a recursive descent Parser, an AST-walking Bytecode Compiler with backpatching, and a stack-based Virtual Machine — all without external dependencies.

The system supports integer and boolean types, arithmetic and comparison operators, variable declarations and assignments, if/else control flow, while loops, and built-in print/input statements. The debug mode provides full visibility into every stage of compilation and execution, making CVM++ an effective educational tool for understanding how programming languages work under the hood.

Future extensions could include string types, function definitions, arrays, a more detailed error reporting system, and an optimizing bytecode pass.

References

[1] Robert Nystrom, *Crafting Interpreters*, 2021. craftinginterpreters.com

[2] Aho, Lam, Sethi, Ullman, *Compilers: Principles, Techniques, and Tools* (Dragon Book), 2nd ed.

[3] ISO C++17 Standard (ISO/IEC 14882:2017)

[4] CMake Documentation, cmake.org